UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

ECE408 - CS483 - CSE408 Spring 2026
Applied Parallel Programming

Lecture 18: Parallel Computation Patterns – Parallel Scan (Prefix Sum)

Volodymyr Kindratenko

3/31/2026

Course Reminders

- Labs
  - Lab 7 due this Friday
- Project
  - Project milestone 2 released, it is due in 2 weeks

- Check your Grades in Canvas!
  - Grades for labs 1-5
  - Grades for Project milestone 1
  - Grades for MT1

Today's Objectives

- To learn parallel scan (prefix sum) algorithms based on reductions
  - Kogge-Stone Parallel Scan
  - Brent-Kung Parallel Scan
  - Hierarchical algorithms
- To learn the concept of double buffering
- To understand tradeoffs between work efficiency and latency

Scan

- A **scan** operation:
  - Takes:
    - An input array $[x_0, x_1, \ldots, x_{n-1}]$
    - An associative operator $\oplus$
      - e.g., sum, product, min, max
  - Returns:
    - An output array $[y_0, y_1, \ldots, y_{n-1}]$ where
      - **Inclusive scan**: $y_i = x_0 \oplus x_1 \oplus \ldots \oplus x_i$
      - **Exclusive scan**: $y_i = x_0 \oplus x_1 \oplus \ldots \oplus x_{i-1}$

Scan Example

- Example:

3	6	7	4	8	2	1	9

↓

3	9	16	20	28	30	31	40

Inclusive Scan

3	6	7	4	8	2	1	9

↓

0	3	9	16	20	28	30	31

Exclusive Scan

- In general:

x_0	x_1	x_2	x_3	x_4	x_5	x_6	x_7

↓

x_0	$x_0..x_1$	$x_0..x_2$	$x_0..x_3$	$x_0..x_4$	$x_0..x_5$	$x_0..x_6$	$x_0..x_7$

Inclusive Scan

x_0	x_1	x_2	x_3	x_4	x_5	x_6	x_7

↓

0	x_0	$x_0..x_1$	$x_0..x_2$	$x_0..x_3$	$x_0..x_4$	$x_0..x_5$	$x_0..x_6$

Exclusive Scan

Sequential Scan

- Sequential scan for sum:

```
output[0] = input[0];
for(i = 1; i < N; ++i) {
    output[i] = output[i-1] + input[i];
}
```

Inclusive Scan

```
output[0] = 0.0f;
for(i = 1; i < N; ++i) {
    output[i] = output[i-1] + input[i-1];
}
```

Exclusive Scan

- In general:

```
output[0] = input[0];
for(i = 1; i < N; ++i) {
    output[i] = f(output[i-1], input[i]);
}
```

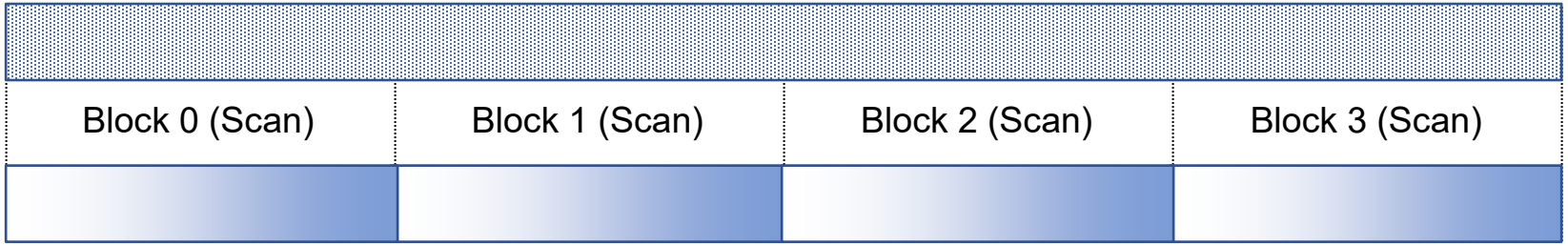
Inclusive Scan

```
output[0] = IDENTITY;
for(i = 1; i < N; ++i) {
    output[i] = f(output[i-1], input[i-1]);
}
```

Exclusive Scan

Segmented Scan

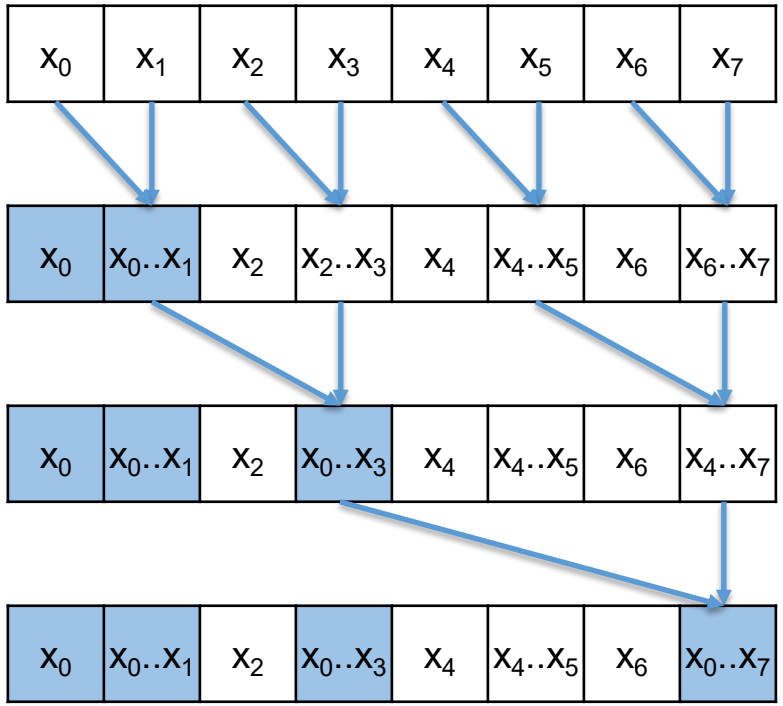
- Parallel scan requires synchronization across parallel workers
- Approach: **segmented scan**
  - Every thread block scans a segment
  - Synchronize across blocks to consolidate the segments (covered later)
- For now, we will focus on implementing a parallel scan in each block



Parallel Inclusive Scan using Reduction Trees

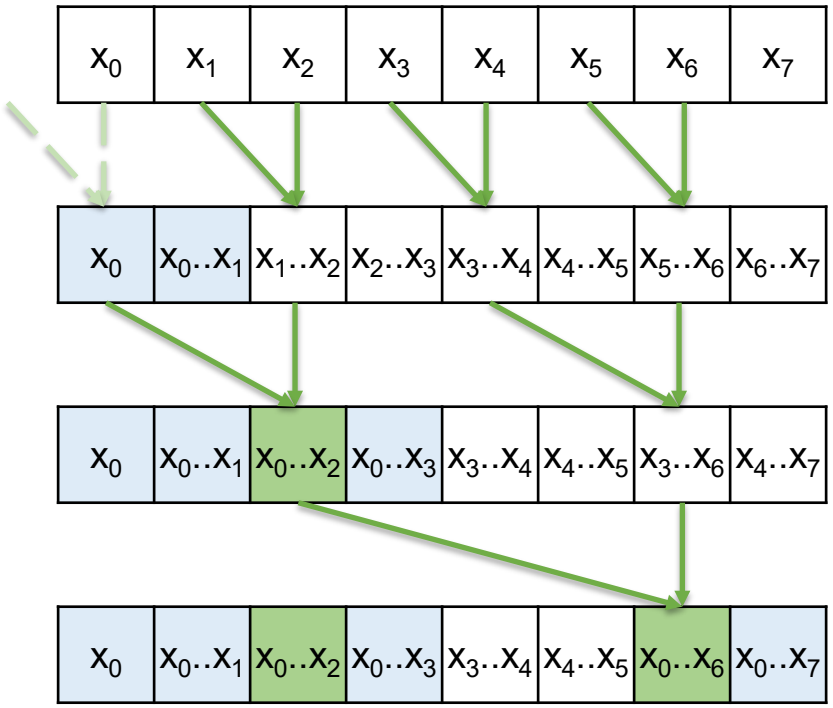
- Calculate each output element as the reduction of all previous elements
  - Some reduction partial sums will be shared among the calculation of output elements
  - Based on hardware added design by Peter Kogge and Harold Stone at IBM in the 1970s – Kogge-Stone Trees
  - Goal: low latency

Parallel (Inclusive) Scan



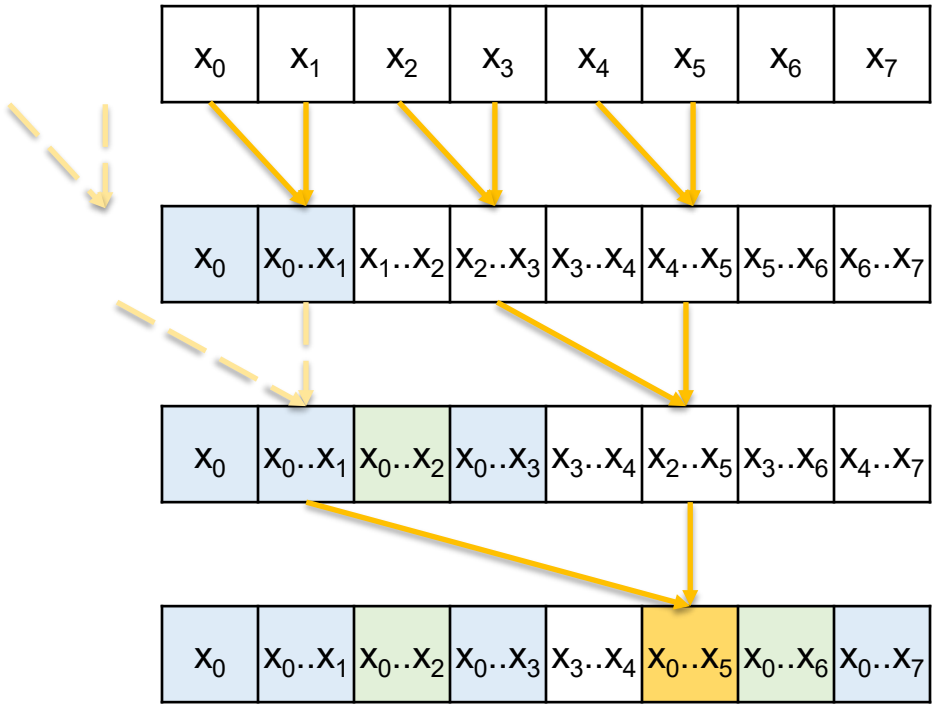
- A parallel reduction tree for the last element gives some others as a byproduct

Parallel (Inclusive) Scan



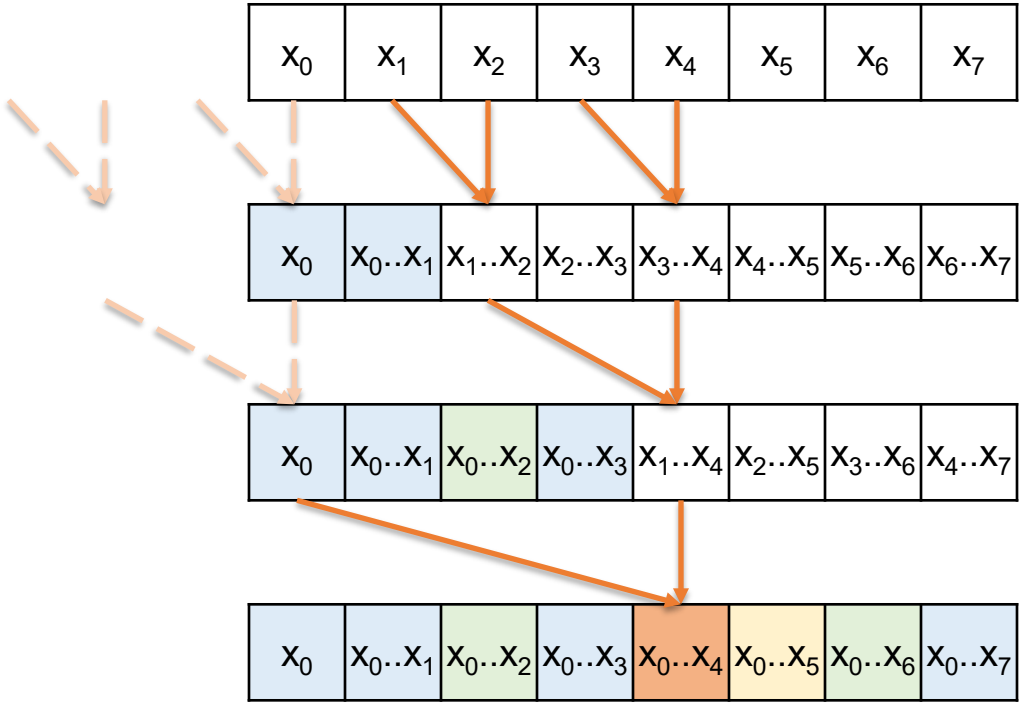
- Another reduction tree gives us more elements

Parallel (Inclusive) Scan



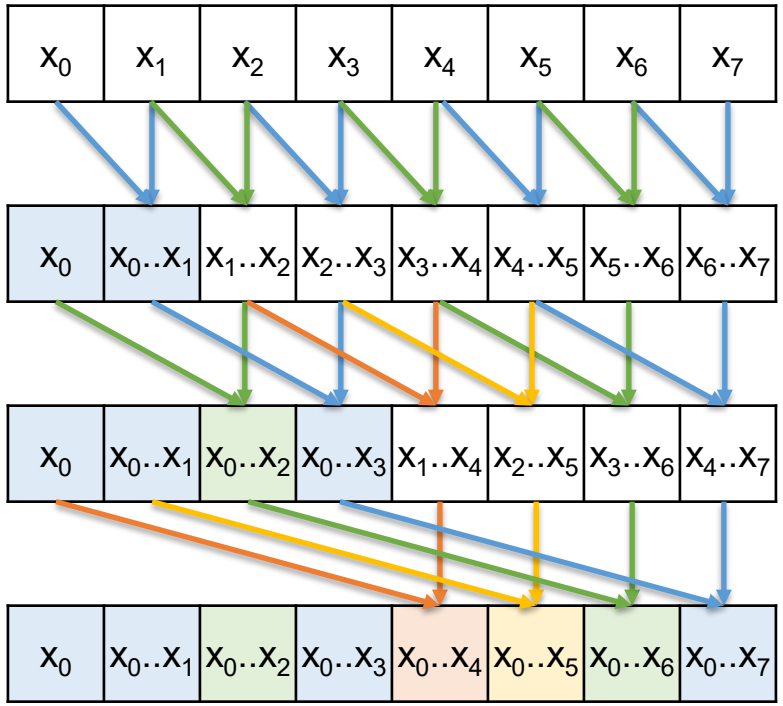
- Keep doing reduction trees until we get all answers

Parallel (Inclusive) Scan



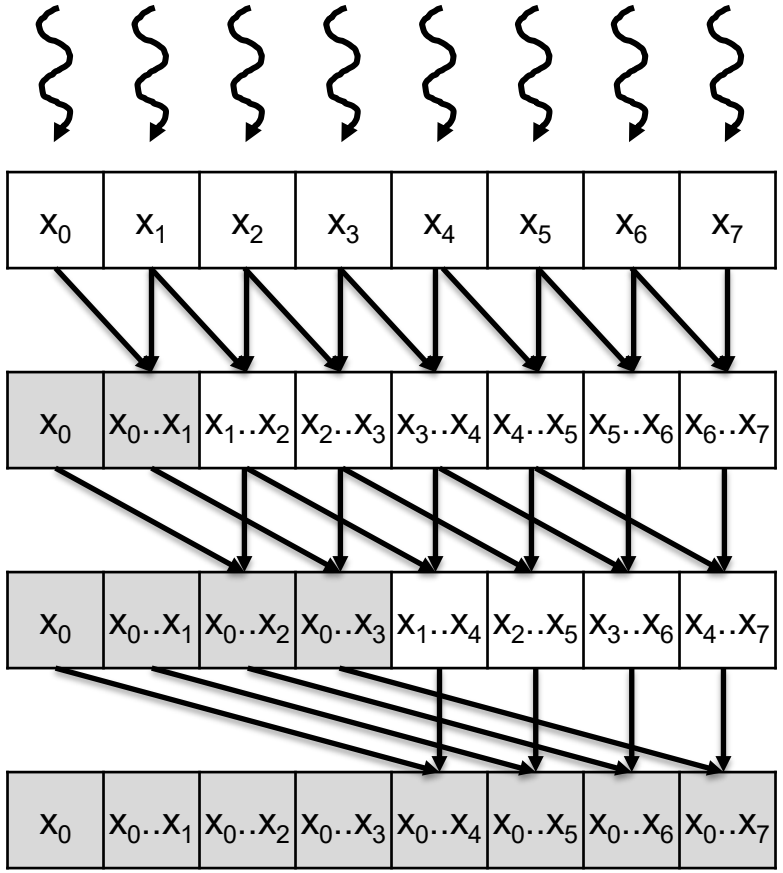
- Keep doing reduction trees until we get all answers

Parallel (Inclusive) Scan



- Overlap the trees and do them simultaneously

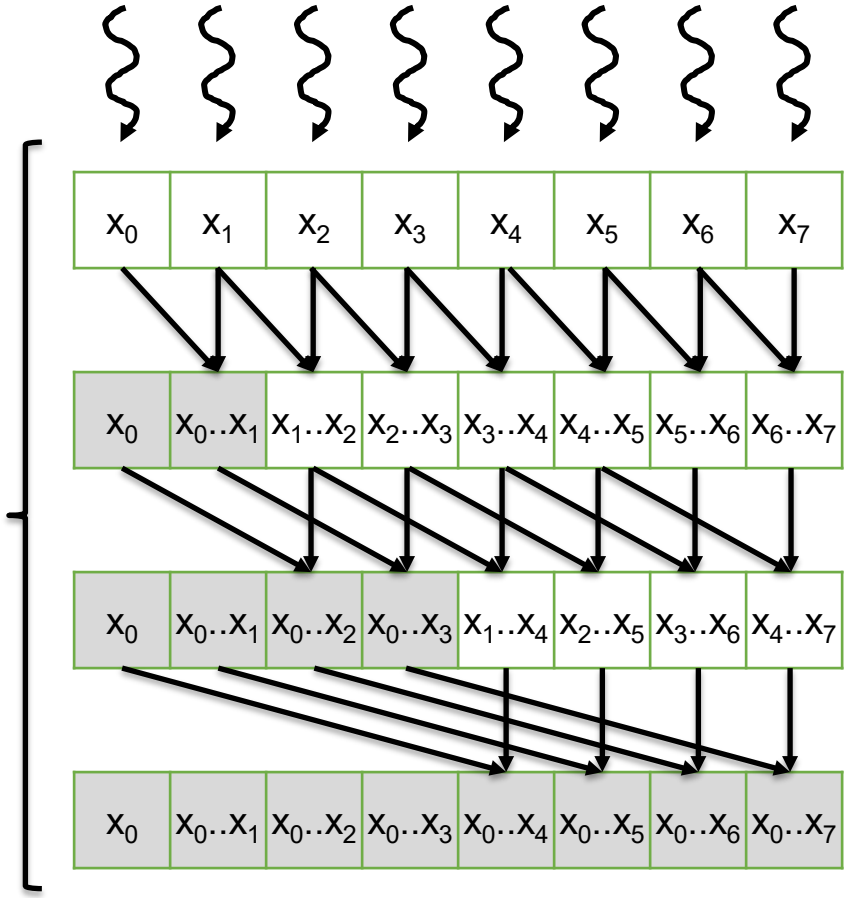
Kogge-Stone Parallel (Inclusive) Scan



One thread for each element

Using Shared Memory

<u>Optimization:</u> load once to a **shared memory** buffer and perform successive reads and writes to the same array can be done in shared memory



One thread for each element

Kogge-Stone Parallel (Inclusive) Scan Code

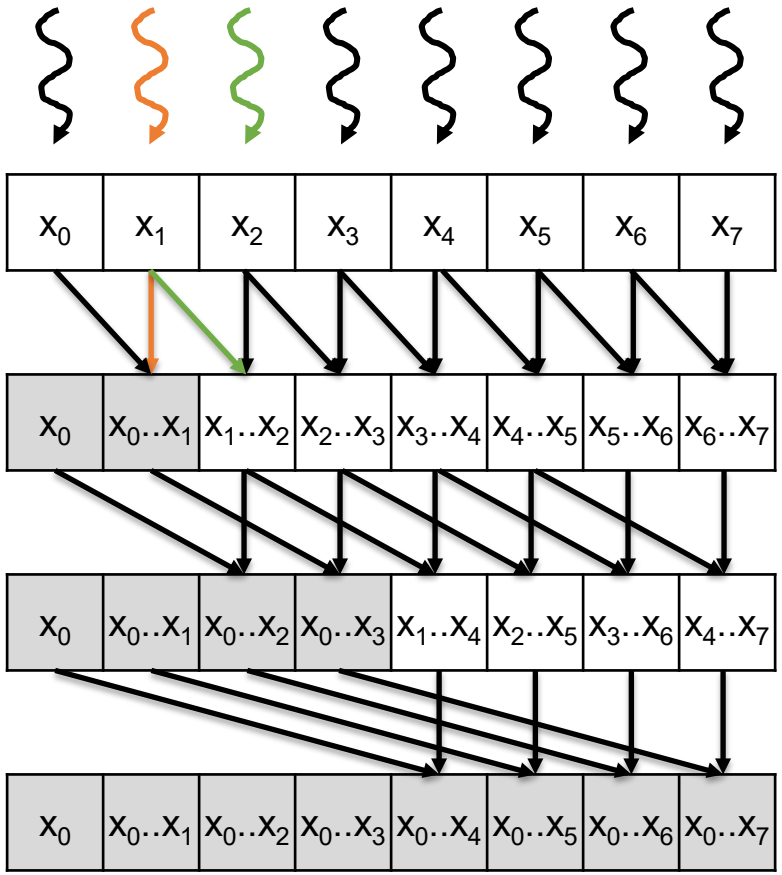
```
unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;

__shared__ float buffer_s[BLOCK_DIM];
buffer_s[threadIdx.x] = input[i];
__syncthreads();

for(unsigned int stride = 1; stride <= BLOCK_DIM/2; stride *= 2) {
    if(threadIdx.x >= stride) {
        buffer_s[threadIdx.x] += buffer_s[threadIdx.x - stride];
    }
    __syncthreads();
}

output[i] = buffer_s[threadIdx.x];
```

Incorrect!
Different threads are reading and writing the same data location without synchronizing

Kogge-Stone Parallel (Inclusive) Scan

Thread 1 may **update value at index 1** before **thread 2 reads it**

Solution: wait for everyone to read before updating



Kogge-Stone Parallel (Inclusive) Scan Code

```
unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;

__shared__ float buffer_s[BLOCK_DIM];
buffer_s[threadIdx.x] = input[i];
__syncthreads();

for(unsigned int stride = 1; stride <= BLOCK_DIM/2; stride *= 2) {
    float v;
    if(threadIdx.x >= stride) {
        v = buffer_s[threadIdx.x - stride];
    }
    __syncthreads();
    if(threadIdx.x >= stride) {
        buffer_s[threadIdx.x] += v;
    }
    __syncthreads();
}

output[i] = buffer_s[threadIdx.x];
```

Wait for everyone to read before writing

Kogge-Stone Parallel (Inclusive) Scan Code

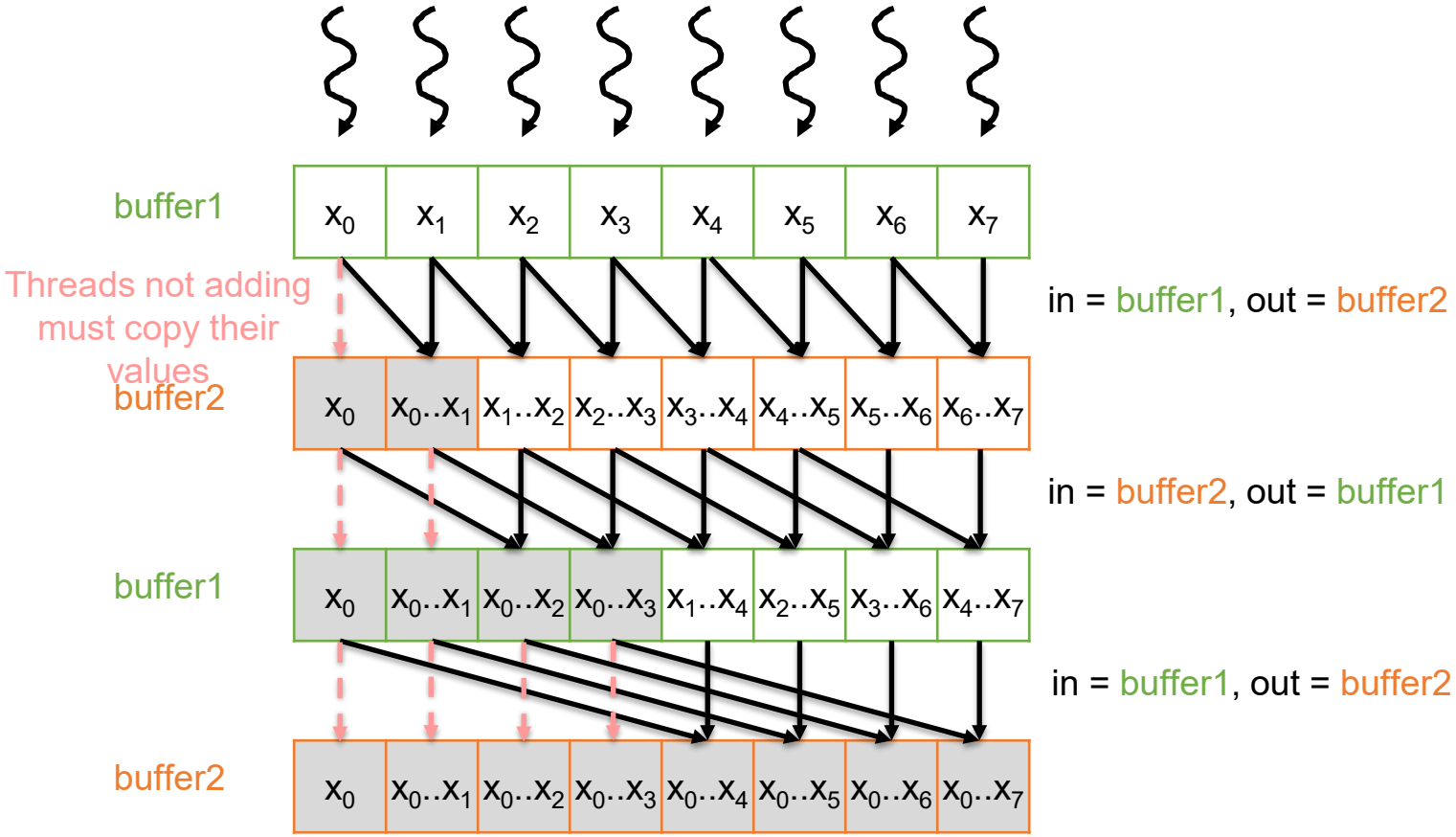
```
unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;

__shared__ float buffer_s[BLOCK_DIM];
buffer_s[threadIdx.x] = input[i];
__syncthreads();

for(unsigned int stride = 1; stride <= BLOCK_DIM/2; stride *= 2) {
    float v;
    if(threadIdx.x >= stride) {
        v = buffer_s[threadIdx.x - stride];
    }
    __syncthreads();
    if(threadIdx.x >= stride) {
        buffer_s[threadIdx.x] += v;
    }
    __syncthreads();
}

output[i] = buffer_s[threadIdx.x];
```

This synchronization enforces a **false dependence** (we only need to finish reading before others write because we are using the same buffer)

This synchronization enforces a **true dependence** (we must finish writing before others can read)

Double Buffering

- **<u>Optimization:</u>** eliminate the synchronization that enforces a false dependence by using separate buffers for reading and writing, and alternate the buffers each iteration (recall: **double buffering**)

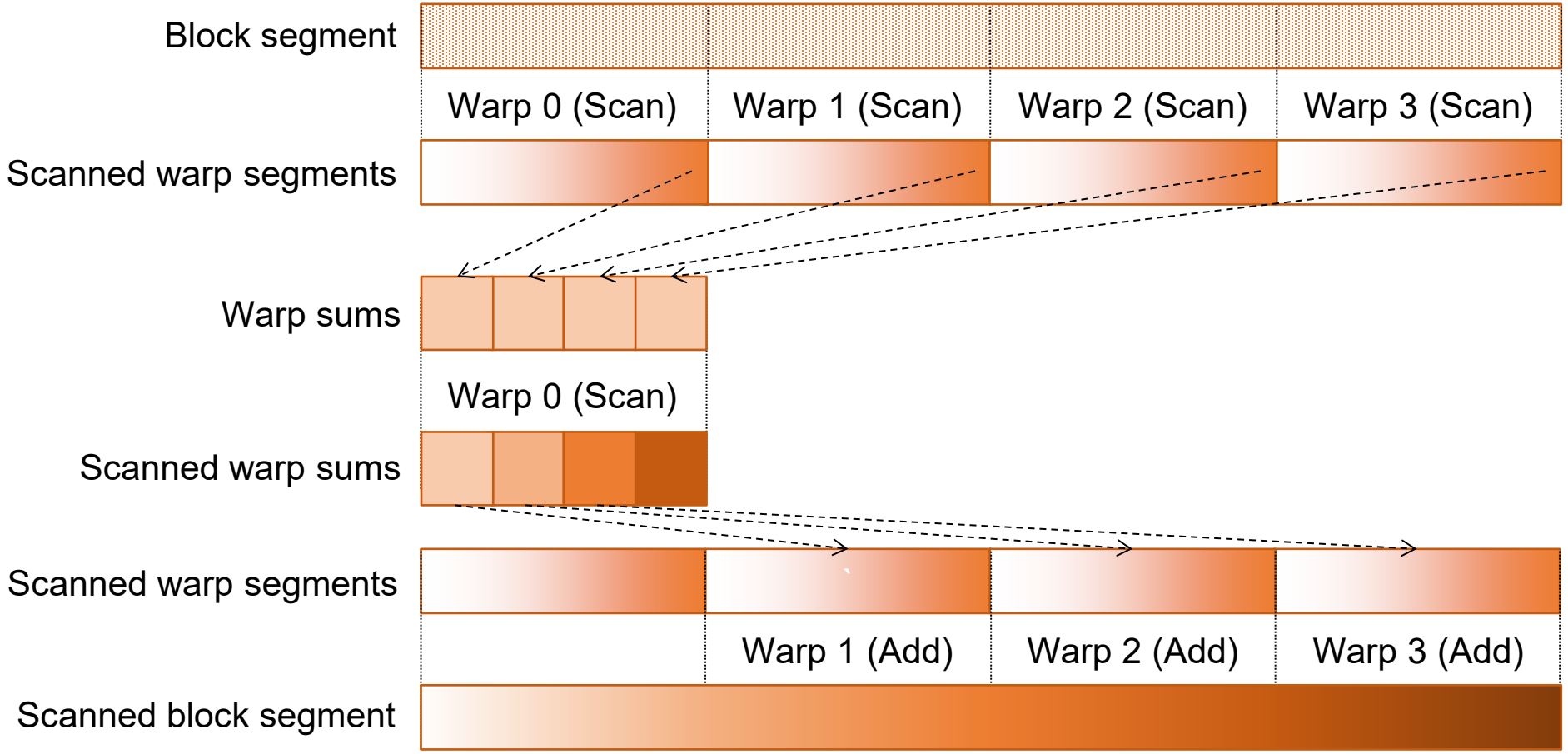


Double Buffering Code

```
unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;

__shared__ float buffer1_s[BLOCK_DIM];
__shared__ float buffer2_s[BLOCK_DIM];
float* inBuffer_s = buffer1_s;
float* outBuffer_s = buffer2_s;
inBuffer_s[threadIdx.x] = input[i];
__syncthreads();

for(unsigned int stride = 1; stride <= BLOCK_DIM/2; stride *= 2) {
    if(threadIdx.x >= stride) {
        outBuffer_s[threadIdx.x] =
                inBuffer_s[threadIdx.x] + inBuffer_s[threadIdx.x - stride];
    } else {
        outBuffer_s[threadIdx.x] = inBuffer_s[threadIdx.x];
    }
    __syncthreads();
    float* tmp = inBuffer_s;
    inBuffer_s = outBuffer_s;
    outBuffer_s = tmp;
}

output[i] = inBuffer_s[threadIdx.x];
```

Reducing Synchronization with Warp-level Primitives

- To avoid block-wide barrier synchronization every iteration, decompose scan across warps and scan warp segments using **warp-level primitives**, then consolidate the result of each warp



Code for Scan with Warp-level Primitives

```
__global__ void scan_kernel(float* input, float* output, unsigned int N) {
    unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
    float val = input[i];
    val = blockScan(val);
    output[i] = val;
}

__device__ inline int laneIdx() { return threadIdx.x % WARP_SIZE; }
__device__ inline int warpIdx() { return threadIdx.x / WARP_SIZE; }

__device__ inline float warpScan(float val) {
    #pragma unroll
    for(unsigned int stride = 1; stride < WARP_SIZE; stride *= 2) {
        float leftVal = __shfl_up_sync(0xffffffff, val, stride);
        if(laneIdx() >= stride) {
            val += leftVal;
        }
    }
    return val;
}
```

Code for Scan with Warp-level Primitives

```
__device__ inline float blockScan(float val) {
    // Warp-level scan
    val = warpScan(val);
    // Collect warp partial sums
    __shared__ float warpSums[NUM_WARPS];
    if(laneIdx() == WARP_SIZE - 1) {
        warpSums[warpIdx()] = val;
    }
    __syncthreads();
    // Scan warp partial sums
    if(warpIdx() == 0) {
        float partialSum = (threadIdx.x < NUM_WARPS)?warpSums[threadIdx.x]:0.0f;
        partialSum = warpScan(partialSum);
        if(threadIdx.x < NUM_WARPS) {
            warpSums[threadIdx.x] = partialSum;
        }
    }
    __syncthreads();
    // Add
    if(warpIdx() > 0) {
        val += warpSums[warpIdx() - 1];
    }
    return val;
}
```

Work Efficiency

- A parallel algorithm is **work-efficient** if it performs the same amount of work as the corresponding sequential algorithm

- Work efficiency of parallel scan
  - Sequential scan performs *N* additions
  - Kogge-Stone parallel scan performs:
    - log(N) steps, N - 2^{step} operations per step
    - Total: (N-1) + (N-2) + (N-4) + … + (N-N/2)

      = N*log(N) - (N-1) = O(N*log(N)) operations
    - Algorithm is not work efficient
      - A factor of log(n) hurts: 20x for 1,000,000 elements!
      - Typically used within each block, where n ≤ 1,024

Improving Efficiency

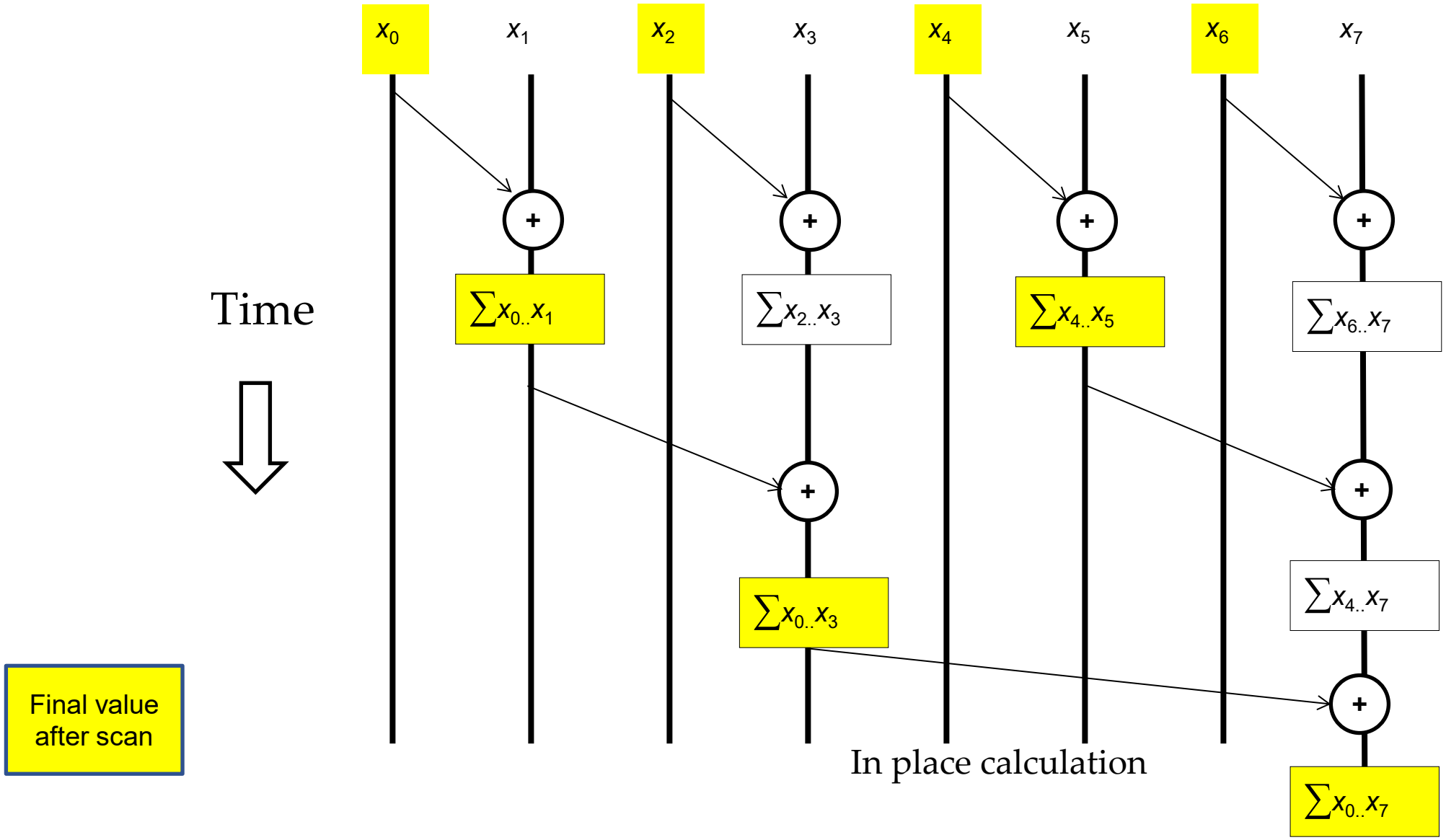
- A common parallel algorithm pattern:

  Balanced Trees

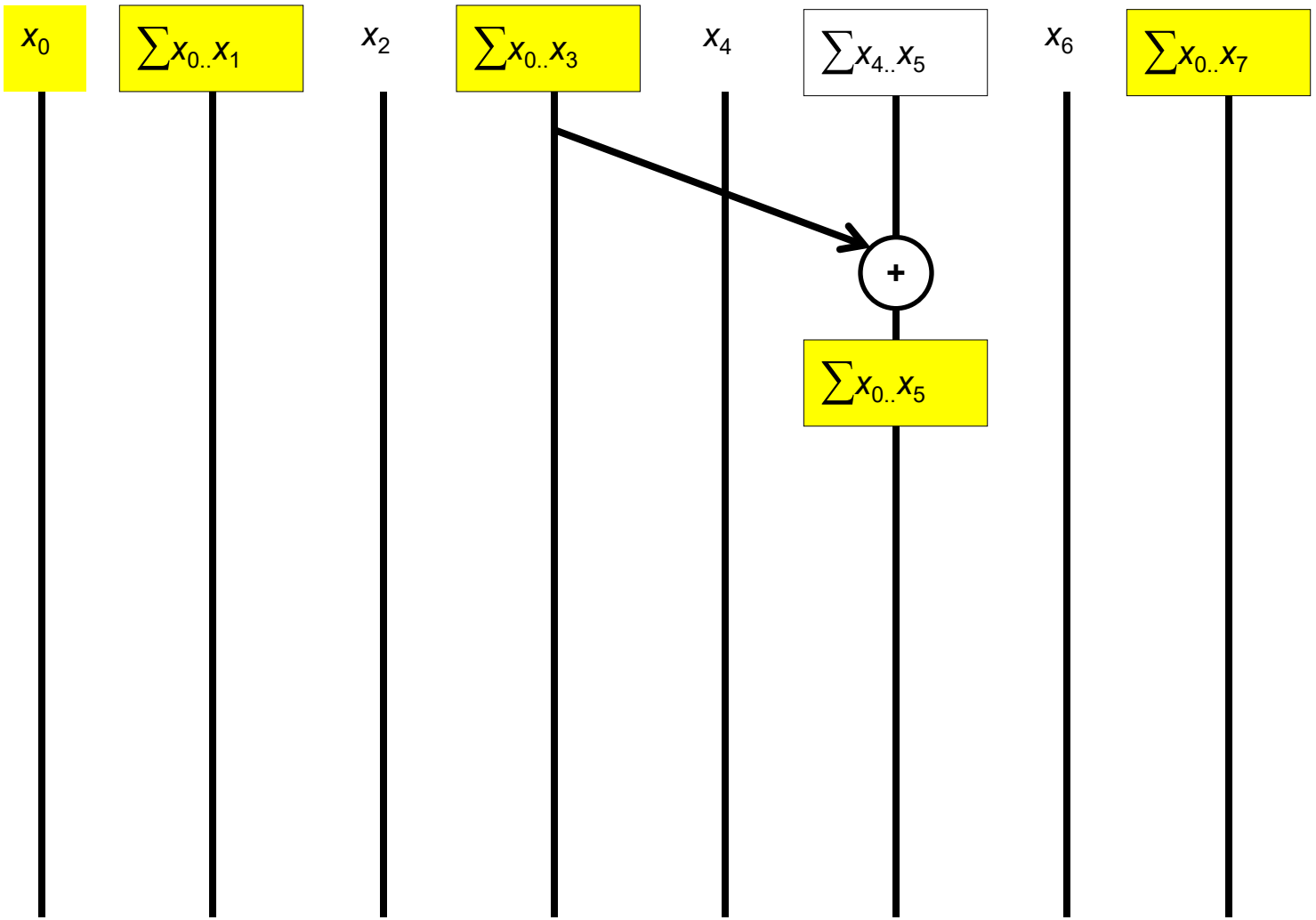
  - Build a balanced binary tree on the input data and sweep it to and from the root
  - Tree is not an actual data structure, but a conceptual pattern

- For scan:
  - Traverse down from leaves to root building partial sums at internal nodes in the tree
    - Root holds sum of all leaves
  - Traverse back up the tree building the scan from the partial sums

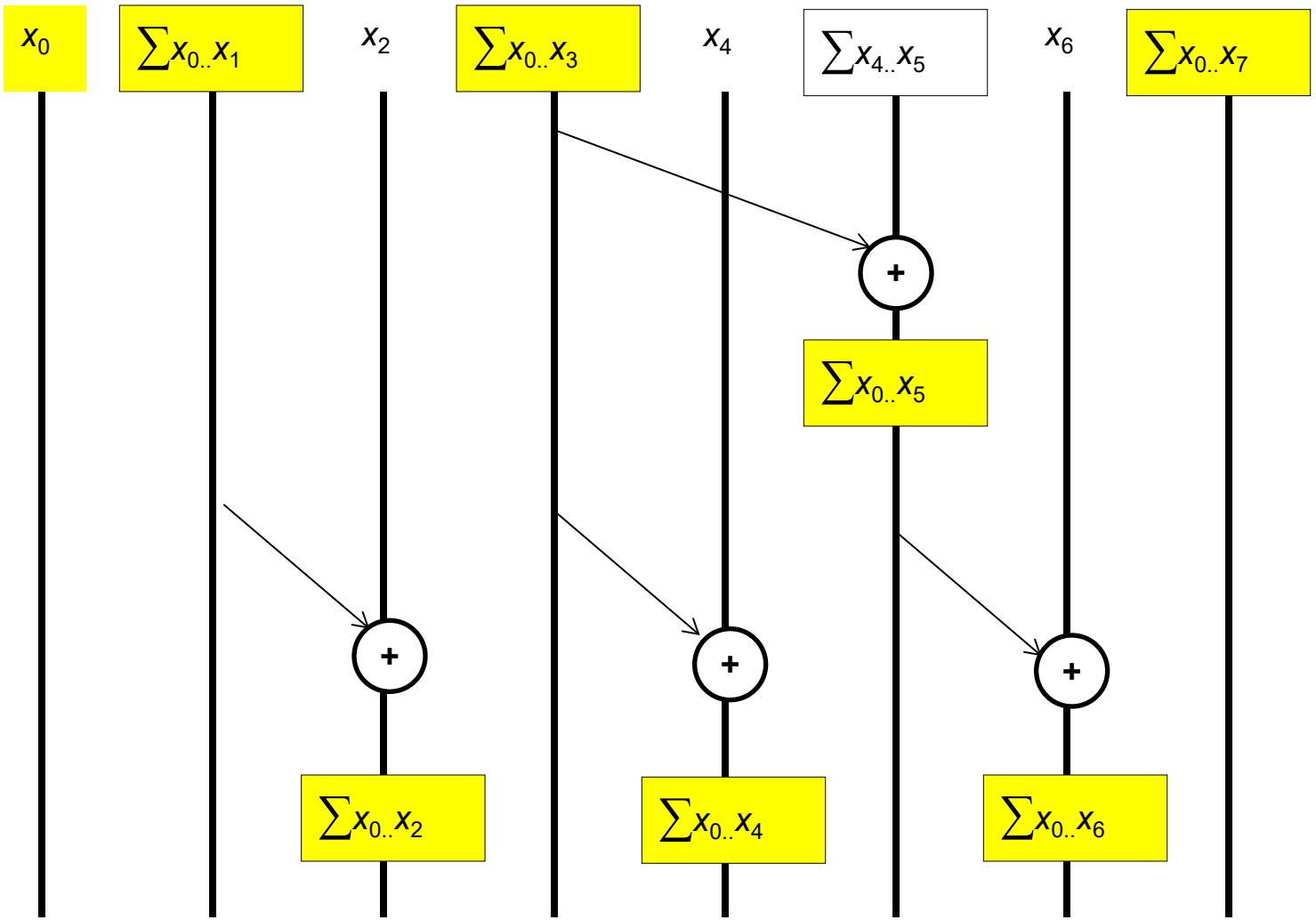
Brent-Kung Parallel Scan Step



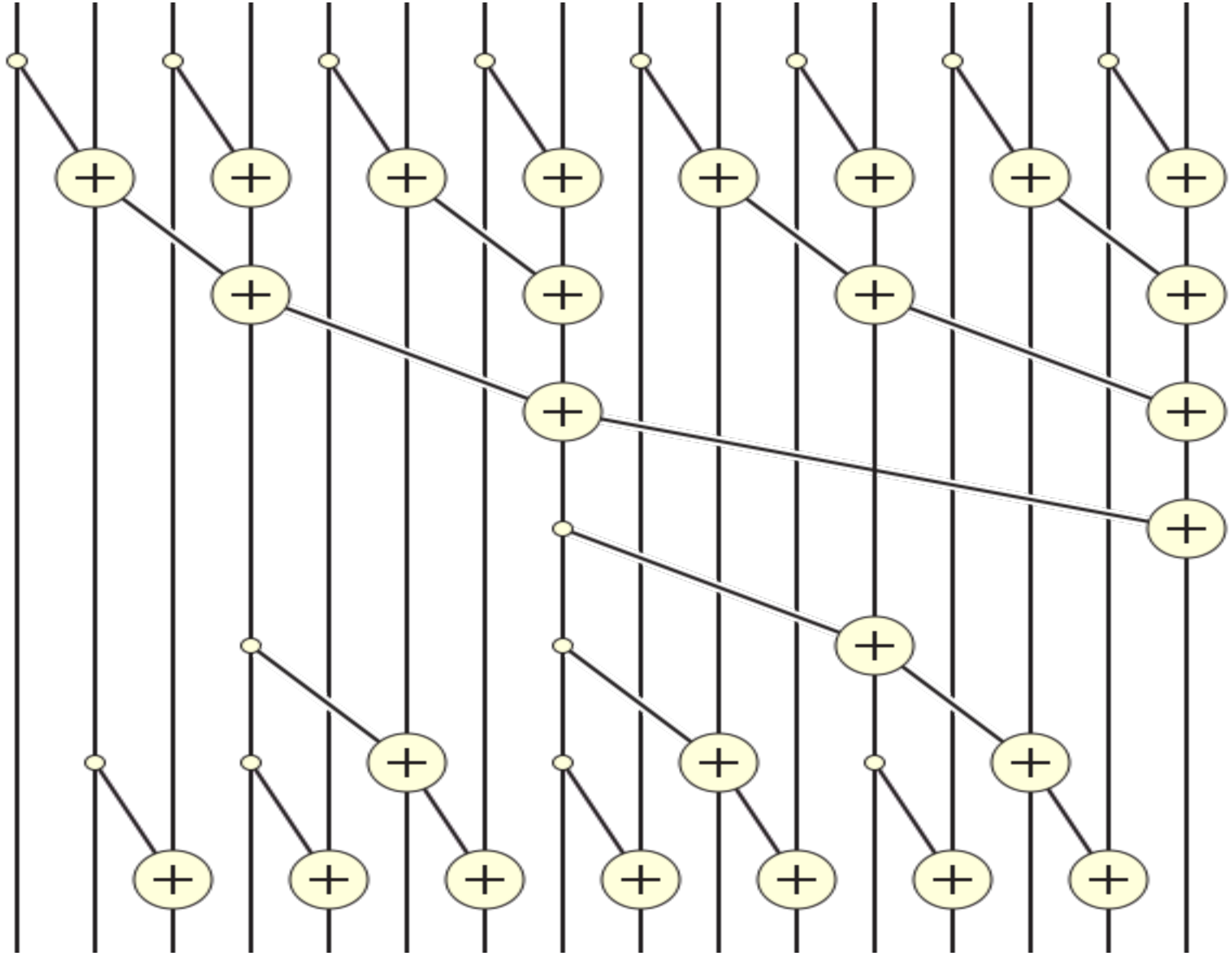
Inclusive Post-Scan Step



Inclusive Post-Scan Step



Putting it Together (Data View)



Reduction Step Kernel Code

```
// float T[2*BLOCK_SIZE] is in shared memory
// for previous slide, BLOCK_SIZE is 8

int stride = 1;
while(stride < 2*BLOCK_SIZE) {
    __syncthreads();
    int index = (threadIdx.x+1)*stride*2 - 1;
    if(index < 2*BLOCK_SIZE && (index-stride) >= 0)
        T[index] += T[index-stride];
    stride = stride*2;
}
```

```
// In our example,
// threadIdx.x+1    = 1, 2, 3, 4, 5, 6, 7, 8
// stride = 1, index = 1, 3, 5, 7, 9, 11, 13, 15
```

Post Scan Step (Distribution Tree)

```
int stride = BLOCK_SIZE/2;
while(stride > 0) {
    __syncthreads();
    int index = (threadIdx.x+1)*stride*2 - 1;
    if ((index+stride) < 2*BLOCK_SIZE)
        T[index+stride] += T[index];
    stride = stride / 2;
 }
```

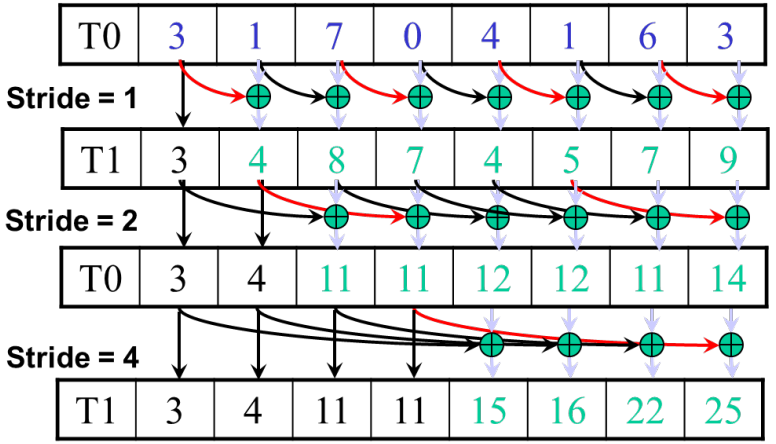
```
// In our example,
// BLOCK_SIZE=8 stride=4, 2, 1
// for first iteration, active thread = 0 index = 7, stride = 11
```

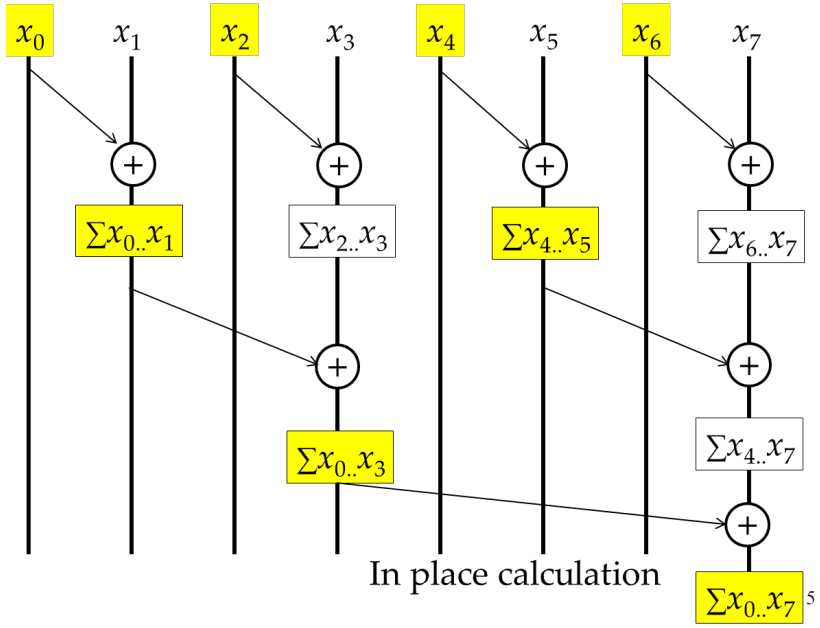
Work Analysis

- The parallel Scan executes 2*log(n) parallel iterations
  - log(n) in reduction and log(n) in post scan
  - The iterations do n/2, n/4,..1, (2-1), …., (n/4-1), (n/2-1) useful adds
  - In our example, n = 16, the number of useful adds is 16/2 + 16/4 + 16/8 + 16/16 + (16/8-1) + (16/4-1) + (16/2-1)
  - Total adds: (n-1) + (n-2) – (log(n) -1) = 2*(n-1) – log(n)→ O(n) work
- The total number of adds is no more than twice of that done in the efficient sequential algorithm
  - The benefit of parallelism can easily overcome the 2× work when there is sufficient hardware

Kogge-Stone vs. Brent-Kung

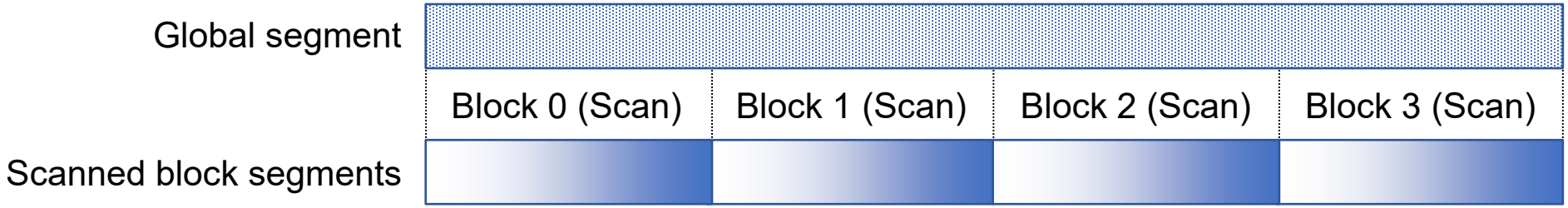
- Brent-Kung uses half the number of threads compared to Kogge-Stone
  - Each thread should load two elements into the shared memory
- Brent-Kung takes twice the number of steps compared to Kogge-Stone
  - Kogge-Stone is more popular for parallel scan with blocks in GPUs





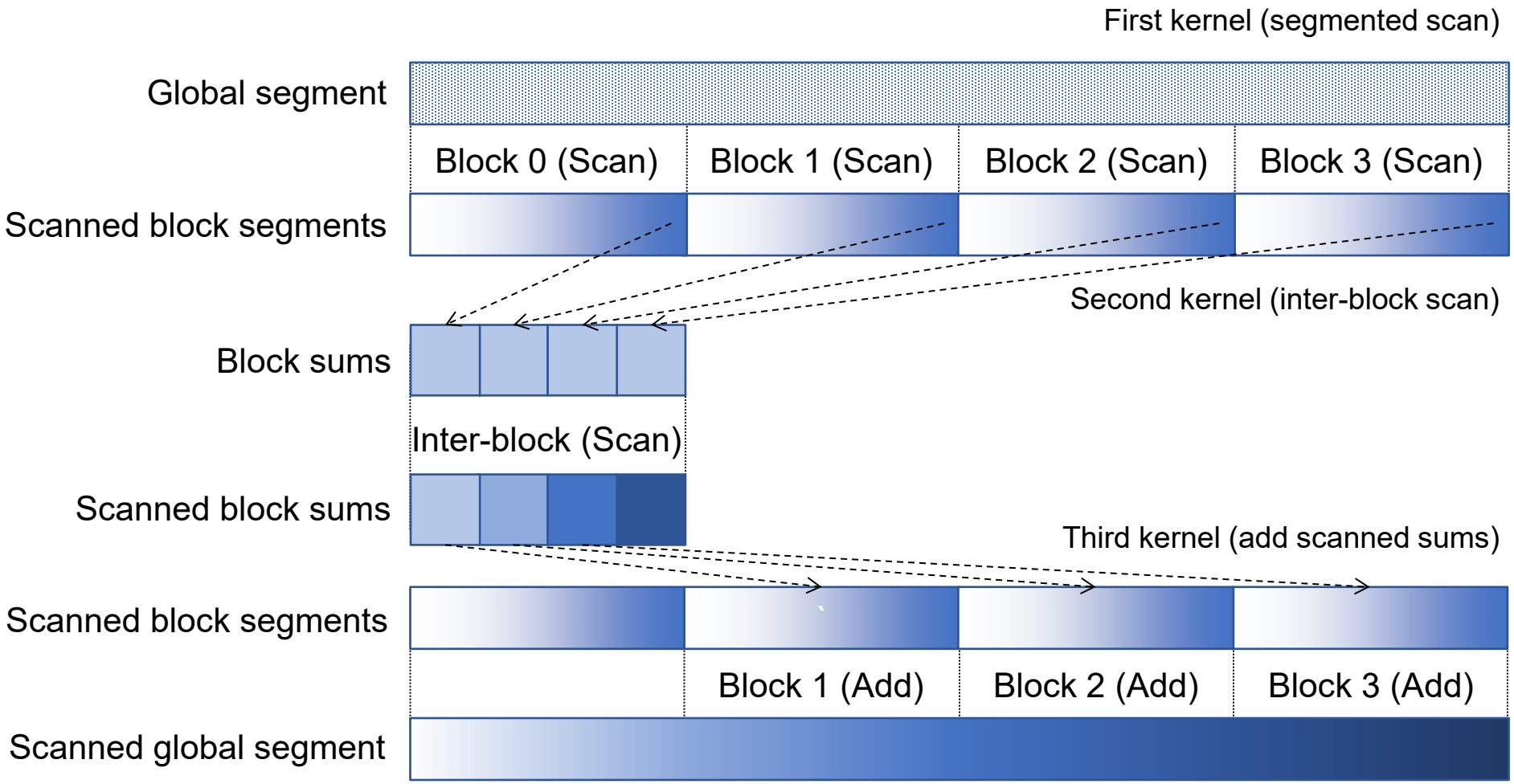
Segmented Scan

- Recall: so far we have performed **segmented scan**



- How do we consolidate the results of the different thread blocks?
  - Try the same strategy as the warp-level and thread-level decomposition
    - Scan partial sums, then add scanned partial sums
  - We can use three separate kernels

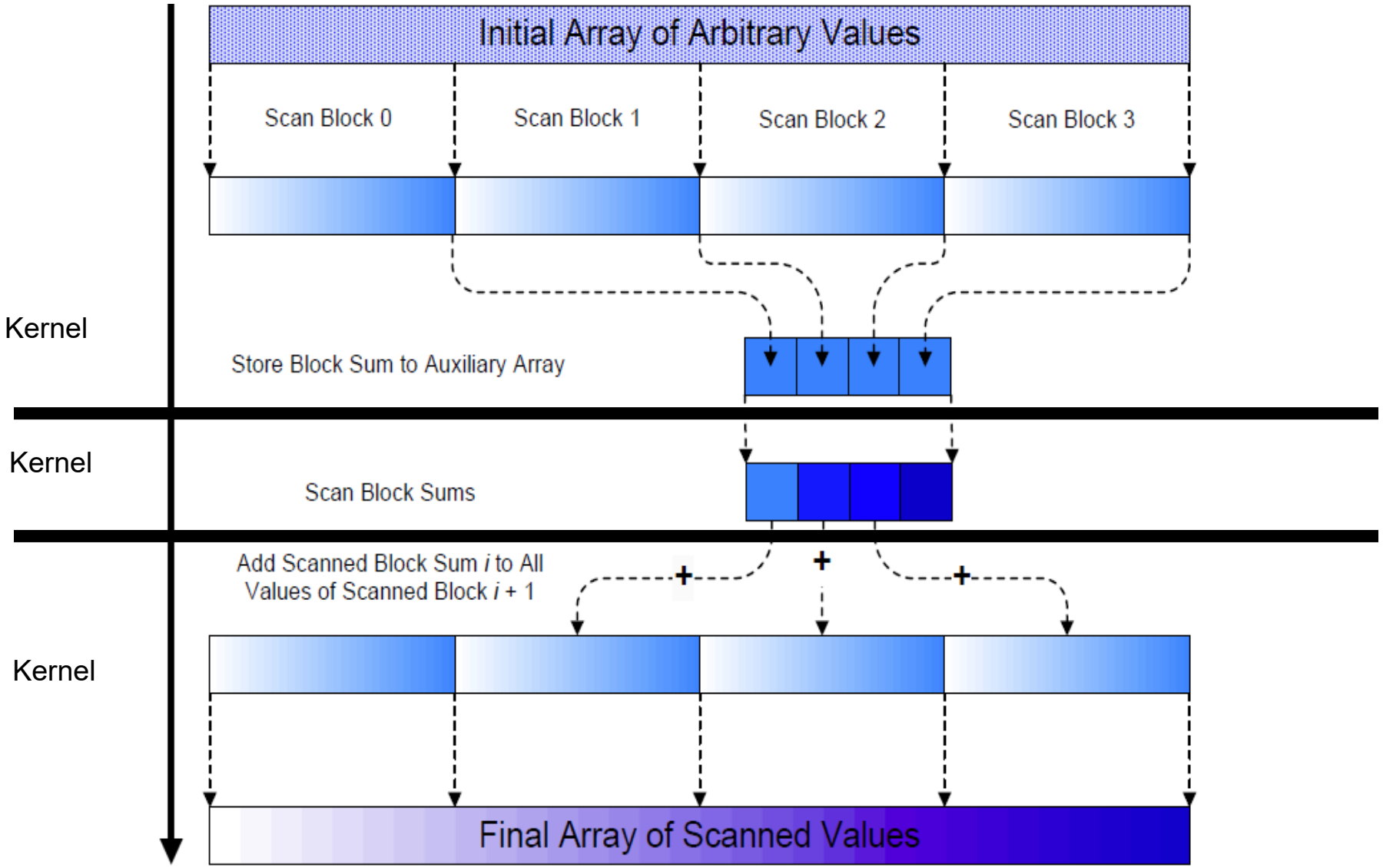
Three-Kernel Scan



Using Global Memory Contents in CUDA

- Data in registers and shared memory of one thread block are not visible to other blocks
- To make data visible, the data has to be written into global memory
- However, any data written to the global memory are not visible until a memory fence. This is typically done by terminating the kernel execution
- Launch another kernel to continue the execution. The global memory writes done by the terminated kernels are visible to all thread blocks.

Overall Flow of Complete Scan



Scan of Arbitrary Length Input

- Build on the scan kernel that handles up to 2*blockDim.x elements from Brent-Kung
  - For Kogge-Stone, have each section of blockDim.x elements assigned to a block
- Have each block write the sum of its section into a Sum array using its blockIdx.x as index
- Run parallel scan on the Sum array
  - May need to break down Sum into multiple sections if it is too big for a block
- Add the scanned Sum array values to the elements of corresponding sections

Memory Bandwidth Considerations

- Scan is memory bound
  - Let's analyze the number of memory accesses for scanning an array of *N* values
  - Ignore accesses to partial sums array which are much fewer for a large block size and coarsening factor
- Three-kernel scan:
  - First kernel: *N* loads + *N* stores = 2*N* memory accesses
  - Second kernel: -
  - Third kernel: *N* loads + *N* stores = 2*N* memory accesses
  - Total: 4*N* memory accesses
- Can we do better?
  - Previous approach is called the **scan-scan-add** approach
  - Another approach is the **reduce-scan-scan** approach, and it can reduce to 3*N* memory accesses

Single-Kernel Scan

- How can we perform the inter-block scan inside the same kernel as the segmented scan?
- One approach is to use **grid-wide barrier synchronizations** with cooperative groups and scan the partial sums with a single thread block
  - Limits the number of thread blocks that can execute, thereby the size of the array that can be scanned
- Another approach is to use **unidirectional synchronization** to pass the partial sums from earlier thread blocks to later thread blocks

Any More Questions?

Things to Read:
Textbook chapter 11

Things to Do:
Lab 7

Problem Solving

- Q: Suppose we have a kernel function that performs partial sum reduction. The block dimension is (64,1,1). During the **second** iteration of the for loop, is there any warp that has a control divergence?

```
__shared__ int partialSum[2*BLOCK_SIZE];
for (int stride = blockDimx.x; stride >= 1; stride = stride/2)  {
   __syncthreads();
   if (threadIdx.x < stride) partialSum[t] += partialSum[t + stride];
}
```

- A:
  - No

Problem Solving

- Q: Consider a kernel that performs Brent-Kung scan algorithm and assume that there are 1024 elements in a section, and a warp size is 32. In which iteration will there be at least one warp that has a control divergence? The stride is 1 for the first iteration.

- A:
  - In 6th iteration

Problem Solving

- Q: Suppose we use Brent-Kung in a hierarchical approach to perform parallel scan on a 1D input array of 2^{42} elements. We use 1024 threads per block in all our Brent-Kung kernels and our GPU supports at most 2048 blocks per grid. Each block processes 2048 elements. What is the best approximation of the number of floating-point add operations performed per thread block in the reduction and post scan steps in the Brent-Kung kernel (summed together)?

- A:
  - 2048 x 2